Software Architecture and Design Specification

Project: Attrition Analysis System (AAS)

Version: 1.1

Authors: H K Manaswin (PES1UG24AM108), Ganesh Aroda C (PES1UG24AM102), Harsha R (PES1UG24AM115), R Harshith (PES1UG24AM117)

Date: 27-10-2026

Status: Final (as built, v1.0 release)

Revision History

Version	Date	Author	Change Summary
0.1	20-10-2026	Project team	Architecture style and components identified
1.0	27-10-2026	Project team	UML diagrams, API design, security architecture added
1.1	07-11-2026	Project team	Updated to the built system: risk model in the main application, in-memory cache, scheduled jobs, /api paths. Full list in docs/DEVIATIONS.md

Approvals

Role	Name	Signature/Date
Course Coordinator		
Project Guide		

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Attrition Analysis System (AAS), describing how the requirements in the AAS SRS v1.0 are realised in components, interfaces, data and runtime interactions.

1.2 Scope

Covers the complete system: authentication and organization setup, employee records, separation management, exit surveys, attrition analytics, flight-risk scoring, alerts and reports, together with integrations to the client HRMS and the email service.

1.3 Audience

Developers, QA engineers, security reviewers, the project guide/instructors and future maintainers.

1.4 Definitions

AAS, HRMS, RBAC, MFA, JWT, PII, ADR (Architecture Decision Record), STRIDE, TLS, DPDP, SPA (Single-Page Application), ORM. Attrition rate = separations / average headcount x 100.

2. Document Overview

2.1 How to use this document

Section 3 gives the architecture (goals, stakeholders, component and class diagrams, chosen pattern with ADRs, technology stack, risks, traceability and security architecture). Section 4 gives the detailed design (sequence diagrams, API design, error handling, UX). Section 5 holds appendices.

2.2 Related Documents

AAS Software Requirements Specification (SRS) v1.0, AAS Software Test Plan (STP) v1.0, and the Requirements Traceability Matrix (RTM) in SRS section 8.

3. Architecture

3.1 Goals & Constraints

- **Goals:** protect sensitive employee data (SO-1..SO-3); accurate attrition figures; dashboard in <= 3 s for 50k employees (NF-001); 99.5 % availability (NF-003); easy to extend with new analyses and HRMS connectors (NF-005); serve many organizations from one deployment.
- **Constraints:** DPDP Act 2023 compliance; small student team and one-semester timeline; open-source stack; HRMS integration only through vendor APIs.

3.2 Stakeholders & Concerns

Stakeholder	Concerns
HR Managers	Accurate data, quick separation entry, meaningful insights
Department Managers	Seeing only their team, early warning of who may leave
Leadership / CHRO	Trustworthy summary metrics and trends, exportable reports
Employees (leavers)	Privacy and anonymity of feedback
Client IT / Security	Data isolation, encryption, audit, integration with HRMS
Developers / Maintainers	Modularity, testability, clear interfaces
Regulators	Compliance with data-protection law

3.3 Component (UML) Diagram

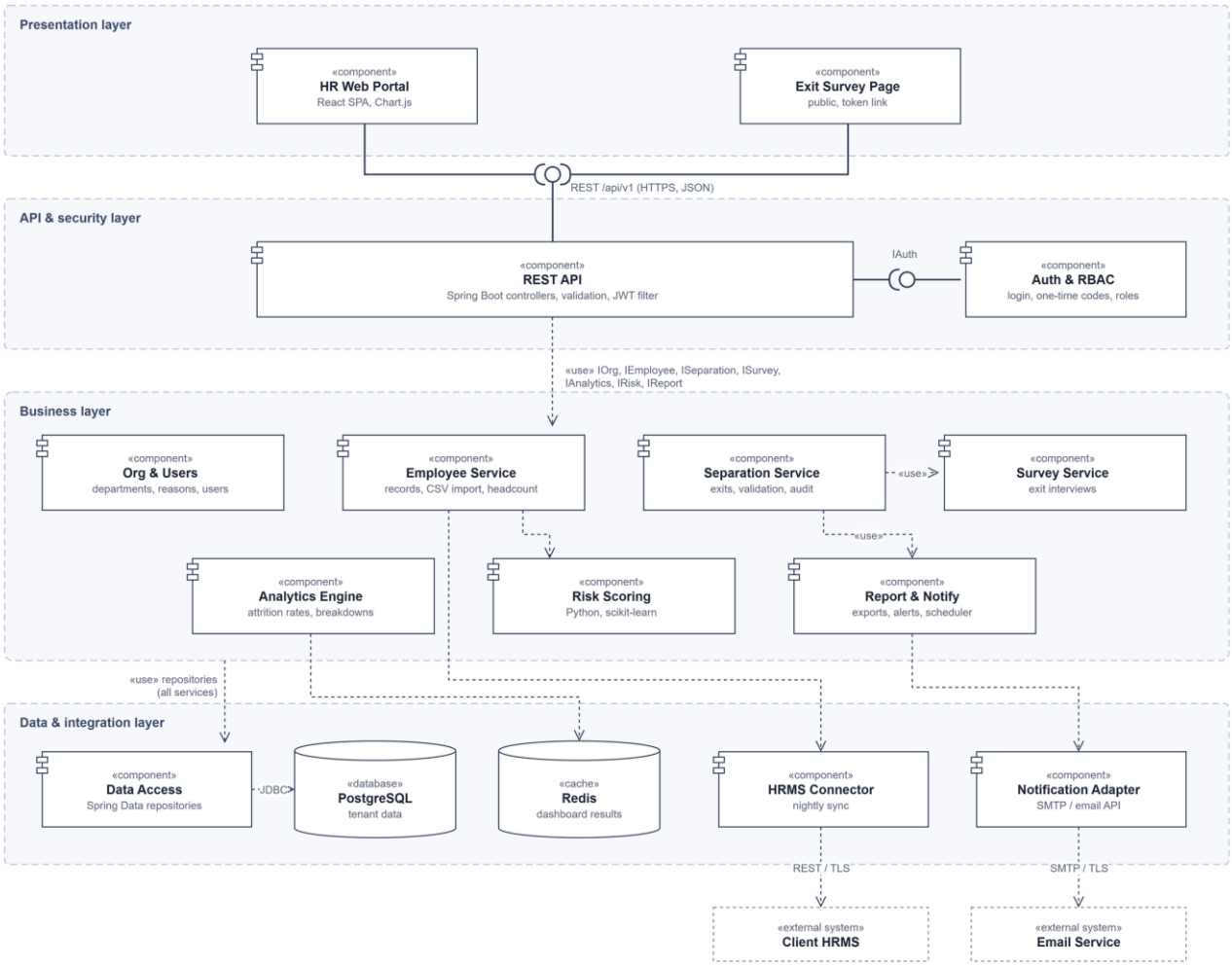


Figure 1 - UML component diagram (layered architecture)

UML Class Diagram (core domain model)

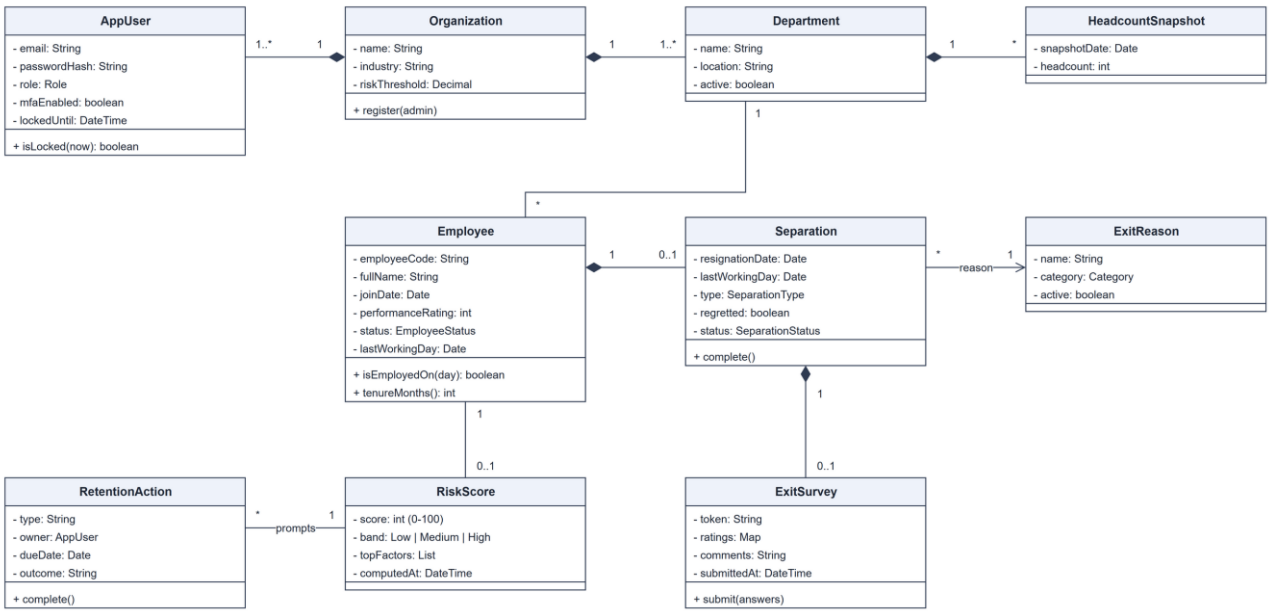


Figure 2 - UML class diagram of the core domain

3.4 Component Descriptions

Component	Responsibility	Key interface
HR Web Portal	React SPA: dashboards, forms, charts (Chart.js); calls REST API only	HTTPS/JSON
Exit Survey Page	Public page opened from the emailed token link; no login	HTTPS
REST API	Spring Boot controllers: routing, request validation, JWT verification	REST /api
Auth & RBAC	Login, one-time codes (TOTP), password hashing, token issue/refresh, role & tenant checks	IAuth
Org & Users	Tenants, departments, exit reasons, thresholds, user accounts	IOrg
Employee Service	Employee CRUD, CSV import, headcount snapshots	IEmployee
Separation Service	Resignations/separations, validation, clearance checklist, audit	ISeparation
Survey Service	Survey token generation, submission, reminders, k-anonymous aggregation	ISurvey
Analytics Engine	Attrition rate, breakdowns, trends, reason ranking; in-memory cache cleared on every data change	IAnalytics
Risk Scoring	Explainable points model (RiskModel); weekly scoring job; top-3 factors; retention actions	IRisk
Report & Notify	CSV export and printable dashboard (PDF), monthly report email, threshold alerts	IReport
HRMS Connector	HrmsClient adapter per vendor (sandbox stub in v1.0); nightly sync with run log	REST / TLS
Notification Adapter	Sends survey links, alerts and reports through the email service	SMTP / TLS
Data Access	Spring Data repositories; every query filtered by organization	JDBC
PostgreSQL	Persistent tenant data (row-level org_id); personal data AES-256-GCM encrypted	SQL

3.5 Chosen Architecture Pattern and Rationale

A layered architecture (Presentation -> API/Security -> Business services -> Data & Integration), deployed as a modular monolith: one Spring Boot application with clearly separated modules, including risk scoring.

- **Why layered:** clear separation of concerns, every request passes through the same security layer (single place for authentication, RBAC and tenant checks), easy for a small team to build, test and deploy.

- **Why risk scoring stayed in the monolith (v1.1):** with no labelled history to train a model, a points model over tenure, time since promotion, rating and team attrition is explainable factor by factor (AAS-F-051) and needs no second service. A trained model can replace RiskModel behind the same interface later.
- **Rejected: full microservices:** too much operational overhead (service discovery, distributed transactions) for the expected scale and team size; modules can be split out later if needed.
- **Rejected: client-server with business logic in the client:** would expose analytics logic and make tenant isolation and anonymity rules hard to enforce.

ADR	Decision	Status
ADR-01	Layered modular monolith + separate Python risk service	Superseded by ADR-05
ADR-02	Shared database with org_id on every table (row-level tenant isolation) instead of DB per tenant	Accepted
ADR-03	Cache dashboard aggregates in Redis for 15 min; invalidate on new separation	Superseded by ADR-06
ADR-04	Exit survey via one-time token link instead of employee accounts	Accepted
ADR-05	Risk scoring as an explainable points model inside the monolith	Accepted
ADR-06	In-memory dashboard cache (5 min) cleared by JPA listeners on any data change	Accepted

3.6 Technology Stack & Data Stores

Layer	Technology
Frontend	React 19, JavaScript, Vite, React Router, Chart.js
Backend	Java 21, Spring Boot 4.1 (Web MVC, Security, Data JPA, Validation, Mail), Flyway migrations
Risk model	Points model in Java (RiskModel), weekly scheduled job
Data stores	PostgreSQL 16 (production, CI); H2 in PostgreSQL mode (development, tests)
Reports	CSV written by the API; PDF via the browser's print of the dashboard
DevOps	GitHub Actions CI (build, tests, JaCoCo, PostgreSQL smoke test, release bundle)
Testing	JUnit 5 + MockMvc, Vitest, JaCoCo, Python load script, Playwright (demo recording)

3.7 Risks & Mitigations

Risk	Mitigation
Poor or missing data from client HRMS causes wrong attrition figures	Validation on import/sync, data-quality report, manual correction screen
Data breach of sensitive employee data	Encryption at rest/in transit, RBAC, audit, regular ZAP scans
Tenant data leak through a missed org_id filter	Central repository filter + automated cross-tenant tests
Biased or inaccurate risk predictions	Exclude protected attributes (gender, age) from model; show factors; review quarterly
Slow dashboards for large organizations	Column projections, in-memory cache, DB indexes; load-tested with 50k employees
Email delivery failure	Send job retries every minute until delivered; survey status visible to HR

3.8 Traceability to Requirements

Requirement(s)	Component(s)
AAS-F-001, F-002, SR-002, SR-004, SR-008	Auth & RBAC Service, API Gateway
AAS-F-003	Org & User Mgmt
AAS-F-010..F-013	Employee Service, HRMS Connector
AAS-F-020..F-023, SR-005	Separation Service
AAS-F-030..F-032, SR-006	Survey Service, Report & Notify Service
AAS-F-040..F-044, NF-001	Analytics Engine, Analytics Cache
AAS-F-050..F-052	Risk Scoring Service, Employee Service
AAS-F-060..F-062	Report & Notify Service

AAS-SR-001, SR-003, SR-007	HSTS, EncryptedStringConverter (AES-256-GCM), request validation

3.9 Security Architecture

Threat modeling (STRIDE) for the main assets: employee PII, separation records, survey responses and risk scores.

Threat	Example in AAS	Mitigation
Spoofing	Stolen HR password used to log in	TOTP MFA for HR roles, lockout (SR-002), short-lived JWT
Tampering	Editing a separation to hide attrition	RBAC, server-side validation, append-only audit log (SR-005)
Repudiation	User denies deleting an employee	Audit log with user, timestamp, before/after values
Information disclosure	Manager identifies who wrote a survey; cross-tenant read	k>=5 aggregation (SR-006), org_id checks (SR-004), AES-256 at rest (SR-003), TLS (SR-001)
Denial of service	Flooding the login or export endpoints	Account lockout; cached dashboard results (rate limiting not built, see DEVIATIONS)
Elevation of privilege	Dept Manager calls HR-only API	Method-level role checks, deny by default

4. Design

4.1 Design Overview

Each business module follows Controller -> Service -> Repository. Controllers handle HTTP and DTO validation, services hold business rules (e.g. attrition formula, separation validation), and repositories access PostgreSQL through JPA, with every query filtered by organization. Long-running work (emails, reminders, scoring, alerts, monthly reports, nightly sync) runs in Spring scheduled jobs.

4.2 UML Sequence Diagrams

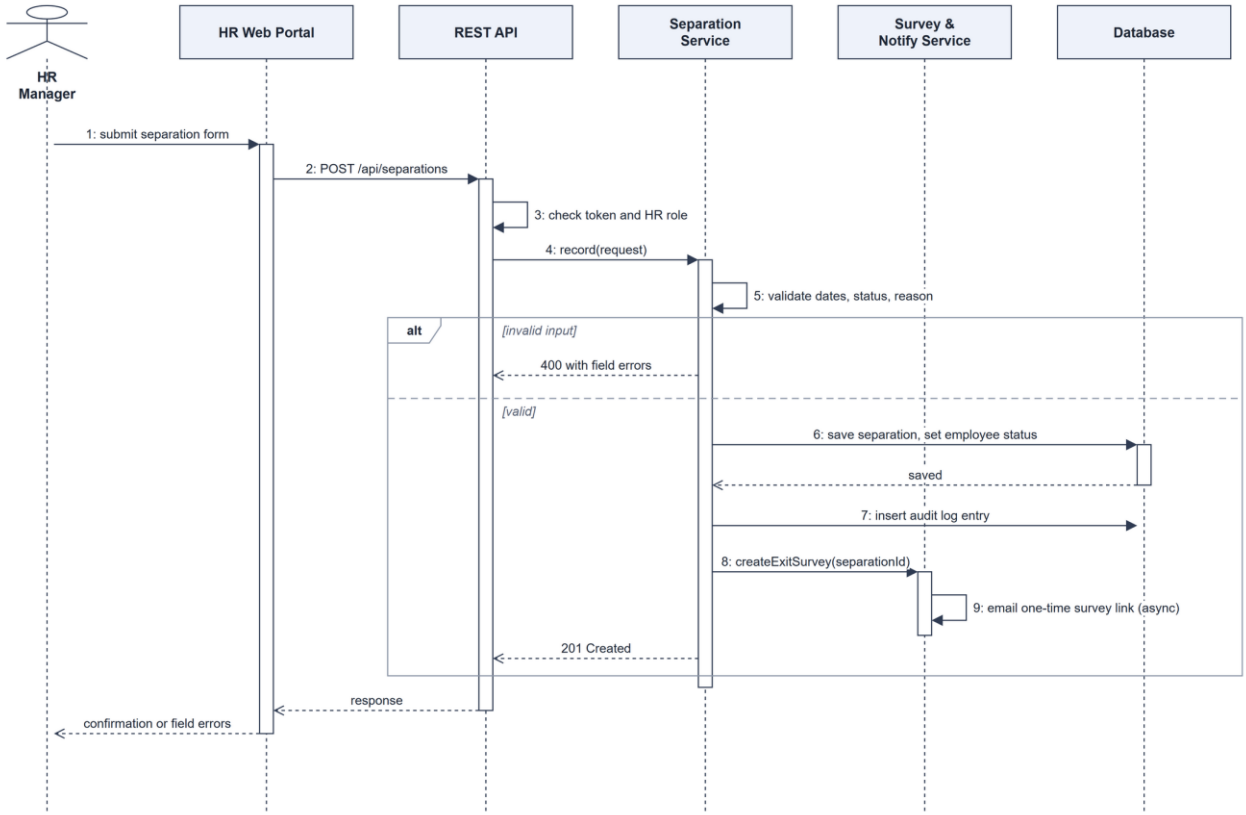


Figure 3 - Sequence diagram 1: record a resignation and trigger the exit survey

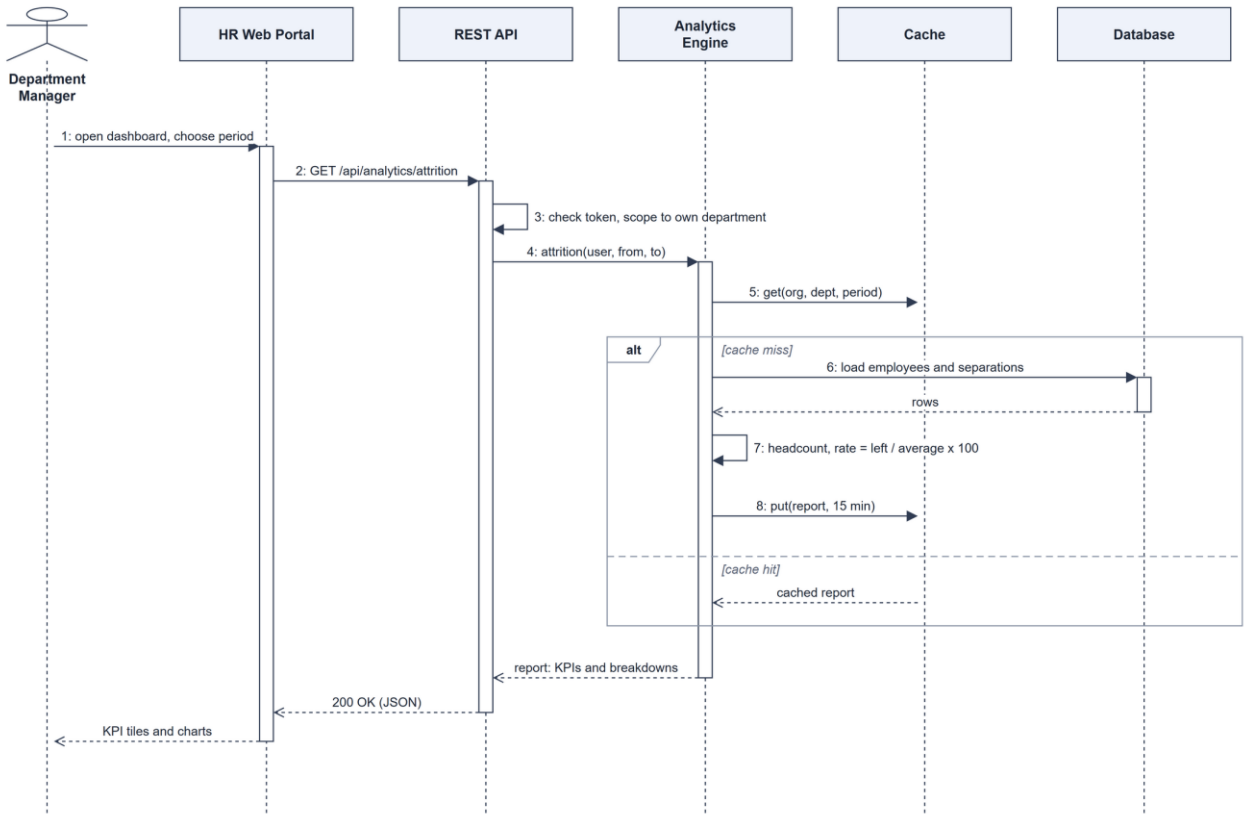


Figure 4 - Sequence diagram 2: view the attrition dashboard (with result caching)

4.3 API Design

All endpoints are under /api, use JSON, and require 'Authorization: Bearer <JWT>' unless stated.

Separation Service - Record separation

Field	Value
Endpoint	/api/separations
Method	POST
Allowed roles	HR_MANAGER
Request	{ "employeeId": "E1001", "resignationDate": "2026-10-15", "lastWorkingDay": "2026-11-14", "type": "VOLUNTARY", "reasonCategory": "COMPENSATION", "regretted": true, "remarks": "..." }
Response	201 Created { "separationId": "c1f...", "status": "NOTICE_PERIOD", "surveyStatus": "SENT" }
Errors	400 validation error (dates/type); 401 not authenticated; 403 role not allowed; 404 employee not found; 409 employee already separated

Analytics Engine - Attrition summary

Field	Value
Endpoint	/api/analytics/attrition?from=2026-07-01&to=2026-09-30&dept=SALES
Method	GET
Allowed roles	HR_MANAGER, DEPT_MANAGER (own dept), LEADERSHIP
Request	query parameters only
Response	200 OK { "avgHeadcount": 240, "separations": 12, "attritionRate": 5.0, "voluntaryRate": 4.17, "regrettedRate": 2.08, "trend": [{"month": "2026-07", "rate": 1.6}, ...], "topReasons": [{"reason": "COMPENSATION", "count": 6, "share": 50.0}] }
Errors	400 invalid date range; 401; 403 department outside user's scope

Risk Scoring Service - Employee risk

Field	Value
Endpoint	/api/employees/{id}/risk
Method	GET
Allowed roles	HR_MANAGER, DEPT_MANAGER (own team)
Request	-
Response	200 OK { "employeeId": "E1001", "score": 78, "band": "HIGH", "computedAt": "2026-10-26", "factors": [{"name": "monthsSincePromotion", "impact": 0.34}, ...] }
Errors	401; 403; 404 employee not found / not scored yet

Survey Service - Submit exit survey

Field	Value
Endpoint	/api/surveys/{token}
Method	POST
Allowed roles	Public (token only)
Request	{ "ratings": {"manager": 2, "pay": 1, "growth": 2, "workLife": 4, "culture": 3}, "comments": "..." }
Response	204 No Content
Errors	404 invalid token; 410 token expired or already used; 400 rating out of range

4.4 Error Handling, Logging & Monitoring

- Uniform error body: { code, message, fieldErrors[], traceId }; no stack traces or internal details returned to clients.
- Structured JSON logs with traceId, orgId and userId; no PII, passwords, tokens or survey text in logs.
- Audit log (append-only table) separate from application logs.
- Monitoring with Prometheus/Grafana: API latency p95, error rate, job failures (sync, email, scoring), cache hit ratio, uptime; alerts to the on-call developer.

4.5 UX Design

- Dashboard first: KPI tiles on top, trend and breakdown charts below, filters persistent across pages.
- Colour-blind-safe chart palette and text labels on every chart; WCAG 2.1 AA contrast.
- Separation form uses dropdowns for type/reason and date pickers with inline validation messages.
- Exit survey: mobile-first, 5 rating questions + 1 comment box, takes under 3 minutes, states clearly that answers are reported anonymously.
- Risk list shows band colour plus the top factors so managers understand why someone is at risk.

4.6 Open Issues & Next Steps

- Choose the first HRMS vendor connector to build (depends on pilot organization).
- Validate the risk model's accuracy on real anonymised historical data.
- Future: sentiment analysis of survey comments, Slack/Teams alerts, mobile app, benchmarking against industry attrition rates.

5. Appendices

5.1 Glossary: AAS, attrition rate, separation, regretted loss, flight risk, tenant, HRMS, RBAC, MFA, JWT, PII, STRIDE, ADR, DPDP.

5.2 References: IEEE 42010 (architecture description), IEEE 29148 (requirements), OWASP Top 10, OWASP ASVS, NIST SP 800-160, Digital Personal Data Protection Act 2023 (India).

5.3 Tools: draw.io (UML diagrams), Swagger/OpenAPI, Postman, GitHub Actions, Jira.